



**POLITECNICO
MILANO 1863**

COMPUTER SCIENCE AND ENGINEERING
NETWORKED SOFTWARE FOR DISTRIBUTED SYSTEMS

A Digital Twin for a Low-Power Wireless Network

Authors

Vivek Kumar Sharma

Shaurya [surname]

Professor

Luca Mottola

Project

#2 — Digital Twin

a.y. 2025/26

1 Introduction

This document describes the design and implementation of a **digital twin** for a low-power wireless sensor network built on **Contiki-NG**, **IPv6/RPL**, and **Akka** actors, coordinated through **Node-RED**. A set of Contiki-NG motes, simulated in Cooja, self-organize into an RPL destination-oriented DAG (DODAG) rooted at a border router and periodically send dummy application traffic toward that root. Separate from this physical network, one Akka actor per IoT node replicates its state — current parent, the last K parents, and the message period T — and is kept synchronized with its physical counterpart in near real time.

The guiding design principle is a **strict separation of concerns** across three layers: the physical network (Contiki-NG) knows nothing about actors or twins; the actor system (Akka) knows nothing about UDP sockets, IPv6 addressing, or Cooja; all cross-layer *coordination* — protocol translation, per-node address tracking, and command routing — lives exclusively in a single, swappable **Node-RED** flow. This means the digital twin’s synchronization policy can be changed by importing a different Node-RED flow, without touching a single line of C or Java.

Requirement Mapping

Table 1 summarizes how each of the four synchronization guarantees required by the assignment is realized, and which component is responsible for it.

Req.	Guarantee	Mechanism
(a)	Parent change → mirrored in twin	Contiki <code>PARENT_CHANGE</code> (UDP) → Node-RED → <code>POST /parent</code> → actor state update
(b)	Application traffic → mimicked in twin	Contiki <code>TRAFFIC</code> (UDP) → Node-RED → <code>POST /traffic</code> → actor event
(c)	Period T change → mirrored to node	Dashboard/actor <code>POST /updateT</code> → Node-RED <code>/set-params</code> → UDP <code>SET_PERIOD</code> to the specific mote
(d)	Crash → mirrored, recovery symmetric	<code>CRASH</code> → actor throws → supervisor restarts twin → Node-RED <code>/revive-node</code> → UDP <code>REVIVE</code> to the same mote

Table 1: Synchronization requirements and their implementation.

2 Architecture

Three-Layer Overview

- **Physical Network Layer (Cooja / Contiki-NG):** four simulated Sky motes. Mote 1 runs the standard `rpl-border-router` firmware and acts as the DODAG root and internet gateway; it holds no digital-twin logic. Motes 2–4 run a custom `iot-node.c` firmware: each resolves its integer id dynamically from `sys/node-id.h`, periodically sends dummy traffic toward the root, detects its own RPL preferred-parent changes, and reacts to two generic UDP commands, `SET_PERIOD` and `REVIVE` (symmetric to a simulated `CRASH`).
- **Coordination Layer (Node-RED):** the only component aware of *both* worlds. It parses raw UDP telemetry, tracks each mote’s current source address, translates events into HTTP calls toward Akka, and translates twin-originated intents back into UDP commands addressed to the correct physical mote.
- **Digital Twin Layer (Akka + Dashboard):** one Akka actor per IoT node holds pure state and reacts only to messages; a lightweight Akka HTTP server exposes REST endpoints and broadcasts every state change over Server-Sent Events (SSE) to a browser-based dashboard (vanilla JS + Canvas) that renders the live topology.

Digital Twin State

Twins are created lazily: a top-level `MoteManager` actor routes every incoming message by `moteId` and, on first contact, spawns a child `ModeActor` for that id via `getContext().actorOf(...)`. Each `ModeActor` holds exactly the state required by the assignment and nothing else:

- `currentParent` — the node’s current RPL parent identifier;
- `parentHistory` — a FIFO of the last $K=3$ *previous* parents (the current one is pushed onto it only when it is actually superseded, never duplicated);
- `periodT` — the current application message period.

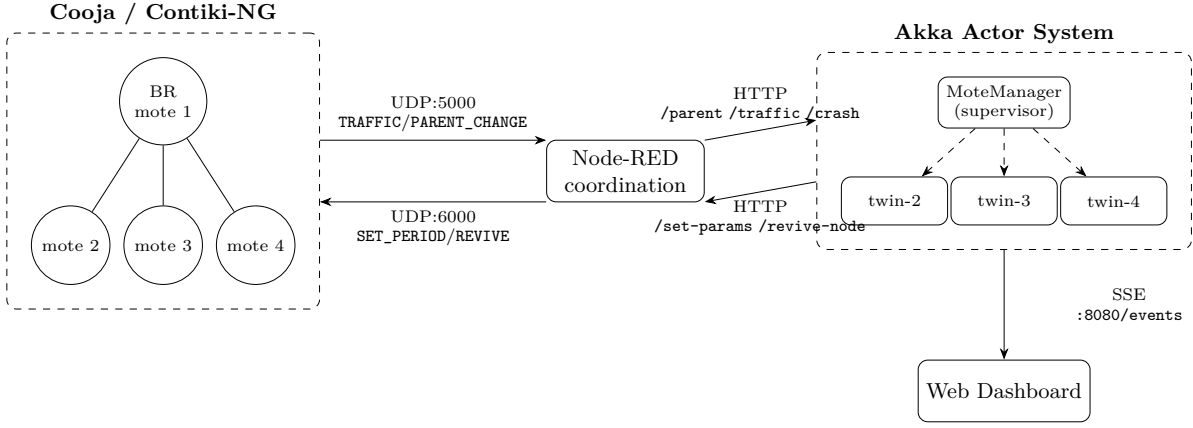


Figure 1: Architecture and message flow: the RPL mesh (left) exchanges UDP telemetry/commands with Node-RED, the sole coordination point, which in turn drives the Akka actor system (center-right) via HTTP; **MoteManager** supervises one twin per mote and restarts it on crash; every state change is broadcast live to the dashboard over SSE.

No actor performs network I/O directly: all outbound side effects (physical period sync, physical revive) are issued by the orchestration layer (**DigitalTwinServer** / **MoteManager**), keeping each twin a pure, easily-testable state machine.

3 Coordination Layer and Crash Recovery

Node-RED as the Only Cross-Layer Component

Per the assignment’s constraint, no IoT-node-specific protocol knowledge exists in Contiki or in Akka: Contiki only understands generic commands (**SET_PERIOD**, **CRASH**, **REVIVE**); Akka only understands actor messages and issues generic HTTP intents (“sync period for mote *X*”, “revive mote *X*”) toward Node-RED. All translation lives in one importable/swappable flow (**nodered.json**):

- An **IP Tracker** function records, for every inbound UDP packet, the sender’s source address keyed by **moteId** in flow context (**node_ip_<id>**). This is what makes commands *addressable*: a **SET_PERIOD** or **REVIVE** request is routed to the specific mote’s last-known address rather than a single fixed target, so concurrent multi-mote operation is correct.
- A **switch** node dispatches **TRAFFIC** and **PARENT_CHANGE** events to distinct HTTP calls (**/traffic**, **/parent**); a **trigger** node acts as a dead-man’s switch, firing **/crash** when a mote has been silent for longer than its expected period, mirroring a physical crash (e.g. a mote stopped in Cooja) even when it cannot itself announce its own death.
- UDP sockets are opened in **dual-stack** (**udp6**) mode, since motes reach Node-RED over a real IPv6 path (**tunslip6** bridging Cooja’s RPL mesh, rooted at **fd00::1**, to the host); an IPv4-only socket would silently drop all genuine mesh traffic while still appearing to work under local loopback testing.

Symmetric Crash Recovery

Requirement (d) demands more than mirroring a crash: recovery must be *symmetric*, i.e. also applied back to the physical node. This is implemented as a closed loop across all three layers:

1. A crash (self-triggered **CRASH** command, or externally detected silence) reaches **ModeActor** as a **MoteCrashed** message; the actor logs the event and **throws** a dedicated **MoteCrashSimulationException**.
2. **MoteManager** applies a **one-for-one** supervisor strategy: only the crashed mote’s own twin is restarted, siblings are unaffected. Restarting recreates the actor from **ModeActor.props()**, resetting **currentParent**, **parentHistory** and **periodT** to a clean state.
3. In the same decision, the supervisor triggers **PhysicalNodeSync**’s revive call, an asynchronous HTTP request to Node-RED’s **/revive-node** endpoint, which resolves the mote’s tracked IP and sends it a **REVIVE** UDP command, resuming its periodic traffic.

The revive trigger is intentionally placed in the supervisor rather than inside **ModeActor** itself, so that twin actors remain free of network side effects and the recovery policy stays entirely inspectable/replaceable at the orchestration layer.

4 Implementation and Verification

Technology Stack

- **Physical layer:** Contiki-NG, RPL-Lite routing, simulated in Cooja (UDGM radio medium) with a native macOS build target for fast iteration.
- **Digital twin layer:** Akka Classic actors (Java 21), Akka HTTP for the REST surface, and an SSE broadcast built on `Source.queue + BroadcastHub` so every connected dashboard receives the same live event stream.
- **Coordination layer:** Node-RED, using only core nodes (`udp in/out`, `http in/request`, `switch`, `trigger`, `function`) so the flow remains portable and inspectable without custom modules.
- **Dashboard:** dependency-free JavaScript and Canvas 2D, with a force-directed layout for the topology view and a built-in demo mode that requires no backend.

Verification

The full pipeline was verified end to end, not just at the unit level, by exercising each synchronization guarantee against a live Node-RED instance, a running Akka process, and UDP peers standing in for physical motes:

- **Parent/traffic mirroring:** injected `PARENT_CHANGE` and `TRAFFIC` packets were observed to reach the correct Akka endpoint and to be broadcast, correctly shaped, over the SSE stream consumed by the dashboard.
- **Per-mote period sync:** a `/updateT` request for one mote was confirmed to produce a `SET_PERIOD` UDP command addressed *only* to that mote’s tracked IP, with a safe fallback when no address has been observed yet.
- **Crash/revive symmetry:** a simulated crash was confirmed to (i) restart the correct twin with clean state, without disturbing sibling twins, and (ii) deliver a `REVIVE` command to the physical/simulated mote, closing the recovery loop required by the assignment.
- **Dual-stack addressing:** both IPv4-loopback and native IPv6 traffic were confirmed to be received and routed correctly by the coordination layer, matching the real `tunslip6`-bridged deployment.

Conclusion

The system satisfies all four synchronization requirements while keeping each layer free of concepts that do not belong to it: Contiki-NG exposes only physical-node behavior, Akka exposes only digital-twin state and supervision, and Node-RED is the single, swappable place where the two are reconciled.